

Module 12 - chapter 21

Automate Provisioning EKS cluster with Terraform - Part 3

Let's check AWS UI new EKS cluster

The screenshot shows the AWS Management Console for the 'myapp-eks-cluster'. The 'Cluster info' section indicates the cluster is 'Active' with a 'Kubernetes version' of 1.36. The 'Node health' section shows 0 issues. The 'Nodes' section lists 3 nodes, all in a 'Ready' state. The 'Node groups' section shows 1 group, 'dev-202606241505578899000000d', with 3 instances.

Node name	Instance type	Compute	Managed by	Created	Status	CPU usage	Memory usage	Ephemeral storage usage
ip-10-0-1-179.eu-west-2.compute.internal	t2.small	Node group	dev-202606241505578899000000d	8 minutes ago	Ready	31%	28%	15%
ip-10-0-2-132.eu-west-2.compute.internal	t2.small	Node group	dev-202606241505578899000000d	8 minutes ago	Ready	31%	28%	15%
ip-10-0-3-140.eu-west-2.compute.internal	t2.small	Node group	dev-202606241505578899000000d	8 minutes ago	Ready	21%	24%	15%

So we have a EKS cluster with one node group called "dev-202606241505578899000000d" and 3 instances created of t2.small type

We can also check the resources to see what is running on the worker nodes

The screenshot shows the 'Resources' tab for the 'myapp-eks-cluster'. The 'Workloads: Pods (11)' section lists 11 pods, all in a 'Running' state. The pods are categorized by namespace: aws-node, coredns, eks-pod-identity-agent, and kube-proxy.

Name	Created	Status
aws-node-hlcws	14 minutes ago	Running
aws-node-qphq5	14 minutes ago	Running
aws-node-tk76v	14 minutes ago	Running
coredns-7cb5bc6b48-rw27m	13 minutes ago	Running
coredns-7cb5bc6b48-z2s92	13 minutes ago	Running
eks-pod-identity-agent-57bfl	14 minutes ago	Running
eks-pod-identity-agent-hzxf5	14 minutes ago	Running
eks-pod-identity-agent-mwvix	14 minutes ago	Running
kube-proxy-6ps9p	13 minutes ago	Running
kube-proxy-f2zw5	13 minutes ago	Running

And we also have configmaps and secrets

Overview **Resources** Compute Networking Add-ons Capabilities Access Observability Update history & backups Tags

Resource types Filter resource types

- Workloads
- Cluster
- Service and networking
- Config and secrets
 - ConfigMaps**
 - Secrets
- Storage
- Authentication
- Authorization
- Policy
- Extensions

Config and secrets: ConfigMaps (11)
An API object used to store non-confidential data in key-value pairs. Pods can consume ConfigMaps as environment variables, command-line arguments, or as configuration files in a volume.

Learn more

Filter namespaces Filter ConfigMaps by name

Advanced query filters

	Name	Created
☐	amazon-vpc-cni	18 minutes ago
☐	aws-auth	16 minutes ago
☐	coredns	15 minutes ago
☐	extension-apiserver-authentication	20 minutes ago
☐	kube-apiserver-legacy-service-account-token-tracking	20 minutes ago
☐	kube-proxy	15 minutes ago
☐	kube-proxy-config	15 minutes ago
☐	kube-root-ca.crt	20 minutes ago
☐	kube-root-ca.crt	20 minutes ago
☐	kube-root-ca.crt	20 minutes ago

We can also see the cluster entry point

Amazon Elastic Kubernetes Service > Clusters > myapp-eks-cluster

myapp-eks-cluster Connect Delete cluster Monitor cluster

Cluster info

- Status: Active
- Kubernetes version: 1.36
- Support period: Standard support until August 2, 2027
- Provider: EKS
- Cluster health: 0
- Upgrade insights: 5
- Node health issues: 0
- Capability issues: 0

Details

API server endpoint
<https://B12689C41D38F9EAD7F85783747EB386y4.eu-west-2.eks.amazonaws.com>

OpenID Connect provider URL
<https://oidc.eks.eu-west-2.amazonaws.com/fd/B12689C41D38F9EAD7F85783747EB386>

Created
27 minutes ago

Cluster ARN
[arn:aws:eks:eu-west-2:706976333940:cluster/myapp-eks-cluster](#)

Platform version
eks.4

EKS Auto Mode
EKS automates routine cluster tasks for compute, storage, and networking to meet application compute needs.

The subnets of the cluster

Overview Resources Compute **Networking** Add-ons Capabilities Access Observability Update history & backups Tags

Networking Manage

VPC
[vpc-03aa0ebd10aa5545f](#)

Cluster IP address family
IPv4

Service IPv4 range
172.20.0.0/16

Subnets
[subnet-0a6865c93f2f8f1e0](#)
[subnet-03050240d427587b4](#)
[subnet-0059dbacf366da42c](#)

Cluster security group
[sg-0171a9b3dfb871388](#)

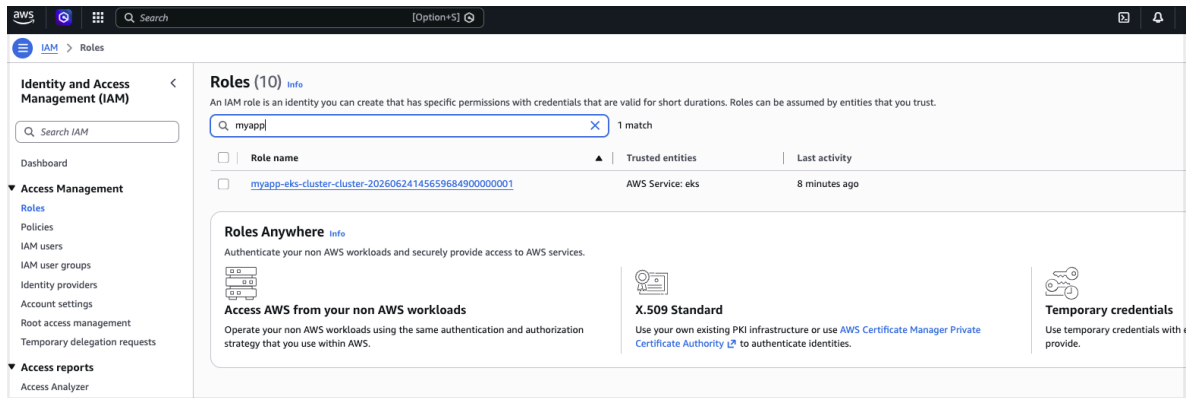
Additional security groups
[sg-03bf4d01a4515c434](#)

API server endpoint access
Public and private

Egress mode
AWS managed

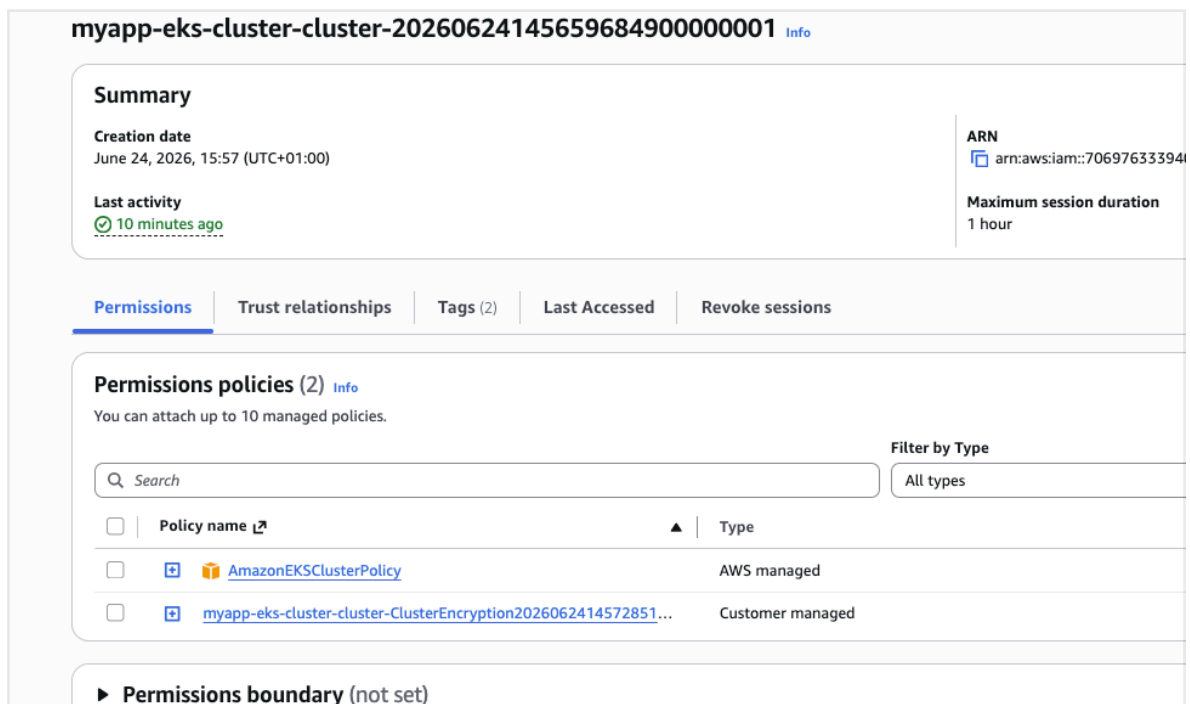
Public access source allowlist
0.0.0.0/0 (open to all traffic)

Now let's check the IAM service and filter by the name of the cluster

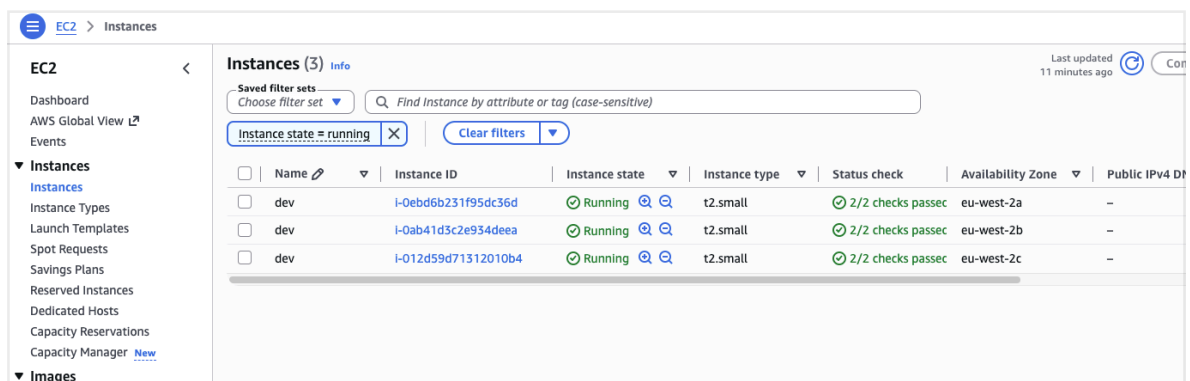


And we can see that the role has been created automatically for the cluster

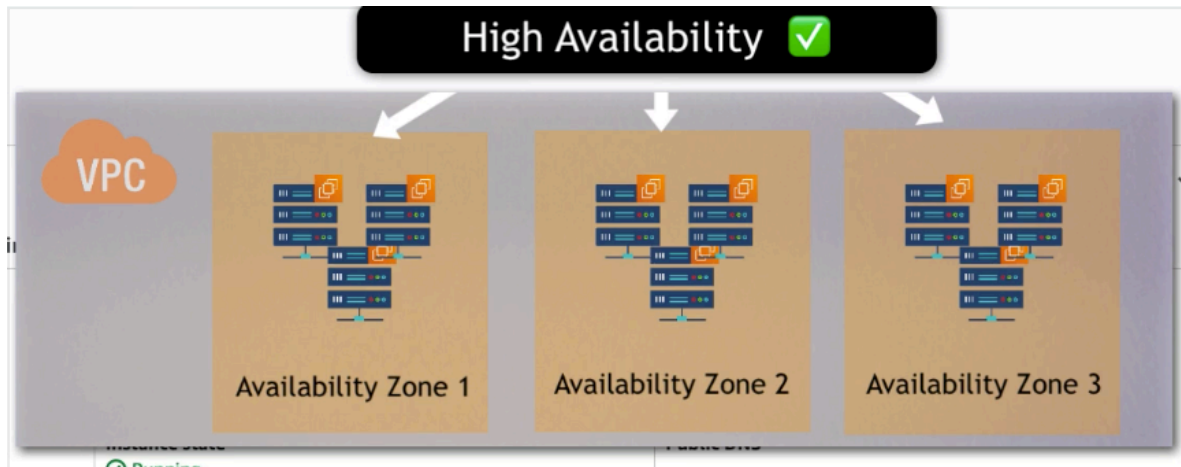
And here are the policies related to the role



Now let's check the EC2 instances



And each EC2 instance is in a different Az



Now let's checkout the VPC service

VPC > Your VPCs

Your VPCs

VPCs VPC encryption controls

Your VPCs (1/2) Info

Find VPCs by attribute or tag

Name	VPC ID	State	Encryption c...	Encryption control ...	Block Public...	IPv4 CIDR	IPv6 CIDR
myapp-vpc	vpc-03aa0ebd10aa5545f	Available	-	-	Off	10.0.0.0/16	-
-	vpc-03677c887cc2f715	Available	-	-	Off	172.31.0.0/16	-

vpc-03aa0ebd10aa5545f / myapp-vpc

Details Resource map CIDRs Flow logs Tags Integrations

Details

VPC ID vpc-03aa0ebd10aa5545f

DNS resolution Enabled

Main network ACL acl-0fc2ac51fa7ae888c

IPv6 CIDR (Network border group) -

Encryption control ID -

State Available

Tenancy default

Default VPC No

Network Address Usage metrics Disabled

Encryption control mode -

Block Public Access Off

DHCP option set dopt-071348549901a183c

IPv4 CIDR 10.0.0.0/16

Route 53 Resolver DNS Firewall rule groups -

DNS hostnames Enabled

Main route table rtb-0beadc10dd47a54c9

IPv6 pool -

Owner ID 706976333940

And we can see the vpc that we created!

And we have 3 route tables

- Default
- Public
- Private

VPC > Route tables

Route tables (3/4) Info

Find route tables by attribute or tag

Name	Route table ID	Explicit subnet associ...	Edge a...	Main	VPC	Owner ID
myapp-vpc-private	rtb-0497d7293626d4cd7d	3 subnets	-	No	vpc-03aa0ebd10aa5545f myapp-vpc	706976333940
myapp-vpc-public	rtb-049e00df32adfc69	3 subnets	-	No	vpc-03aa0ebd10aa5545f myapp-vpc	706976333940
-	rtb-0118aea5c62fb9ff	-	-	Yes	vpc-03677c887cc2f715	706976333940
myapp-vpc-default	rtb-0beadc10dd47a54c9	-	-	Yes	vpc-03aa0ebd10aa5545f myapp-vpc	706976333940

The public route table has the IGW - Internet gateway

Route tables (1/4) [info](#)

Find route tables by attribute or tag

Name	Route table ID	Explicit subnet associ...	Edge a...	Main	VPC	Owner ID
☐ myapp-vpc-private	rtb-0497d293626d4cd7d	3 subnets	-	No	vpc-03aa0ebd10aa5545f myapp-vpc	706976333940
☒ myapp-vpc-public	rtb-049e00df32adfc69	3 subnets	-	No	vpc-03aa0ebd10aa5545f myapp-vpc	706976333940
☐ -	rtb-0118aea5c82fb9ff	-	-	Yes	vpc-03677c88f7cc2f715	706976333940
☐ myapp-vpc-default	rtb-0beadc10d647a54c9	-	-	Yes	vpc-03aa0ebd10aa5545f myapp-vpc	706976333940

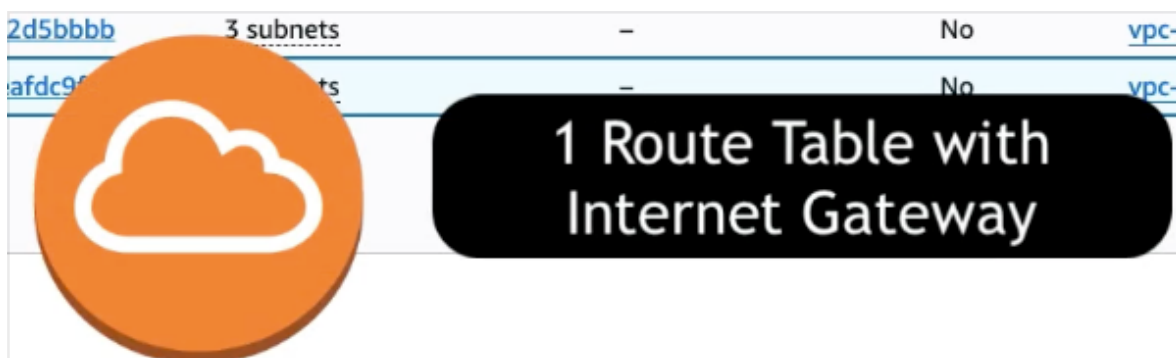
rtb-049e00df32adfc69 / myapp-vpc-public

Details **Routes** Subnet associations Edge associations Route propagation Tags

Routes (2)

Filter routes

Destination	Target	Status	Propagated	Route Origin
0.0.0.0/0	igw-063b5ce750138f166	Active	No	Create Route
10.0.0.0/16	local	Active	No	Create Route Table



And the private route table has a NAT routing

Route tables (1/4) [info](#)

Find route tables by attribute or tag

Name	Route table ID	Explicit subnet associ...	Edge a...	Main	VPC	Owner ID
☒ myapp-vpc-private	rtb-0497d293626d4cd7d	3 subnets	-	No	vpc-03aa0ebd10aa5545f myapp-vpc	706976333940
☐ myapp-vpc-public	rtb-049e00df32adfc69	3 subnets	-	No	vpc-03aa0ebd10aa5545f myapp-vpc	706976333940
☐ -	rtb-0118aea5c82fb9ff	-	-	Yes	vpc-03677c88f7cc2f715	706976333940
☐ myapp-vpc-default	rtb-0beadc10d647a54c9	-	-	Yes	vpc-03aa0ebd10aa5545f myapp-vpc	706976333940

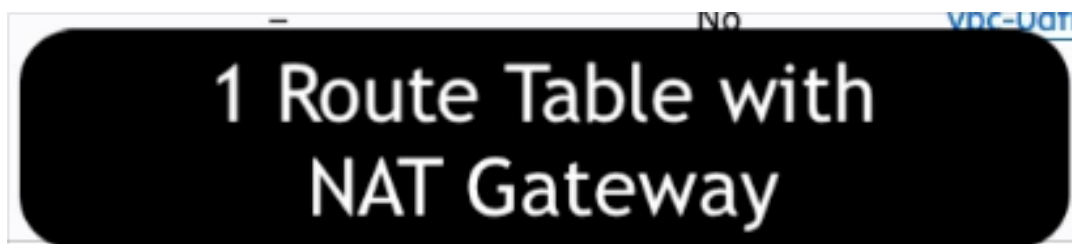
rtb-0497d293626d4cd7d / myapp-vpc-private

Details **Routes** Subnet associations Edge associations Route propagation Tags

Routes (2)

Filter routes

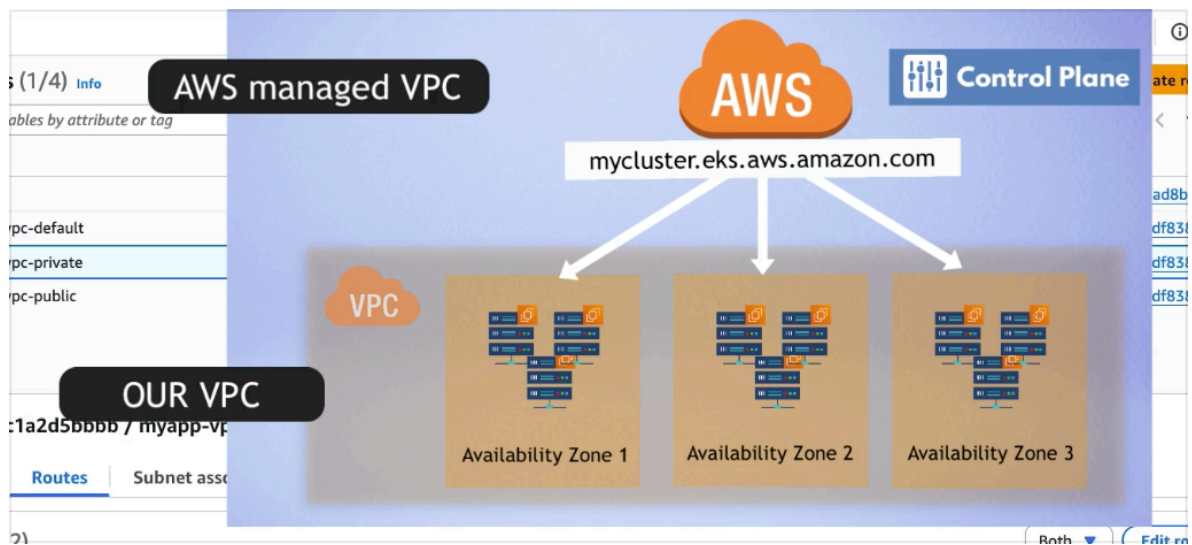
Destination	Target	Status	Propagated	Route Origin
0.0.0.0/0	nat-0878ca6152d3cbe2	Active	No	Create Route
10.0.0.0/16	local	Active	No	Create Route Table



Nat gateway -> This is a route that allows worker nodes to connect to the

control plane nodes.

Control plane nodes are in one VPC (in an account managed by AWS not me) and the worker nodes in another VPC



So with the nat gateway the resources can talk to each other privately from different VPCs.

Now let's check the subnets

The screenshot shows the AWS Subnets console. The table lists 9 subnets. The first 6 subnets are associated with the VPC 'vpc-03aa0ebd10aa5545f' (myapp-vpc-...). The last 3 subnets are associated with the VPC 'vpc-03677c88f7cc2715' (vpc-0371c426ad8b0bb94).

Name	Subnet ID	State	VPC	Block Public	IPv4 CIDR	IPv6 CIDR	IPv6	Avail...	Availability Zone
myapp-vpc-...	subnet-0f052bce470d615a0	Available	vpc-03aa0ebd10aa5545f myap...	Off	10.0.6.0/24	-	-	251	euw2-az1 (eu-west-2c)
myapp-vpc-...	subnet-0b798624884354b62	Available	vpc-03aa0ebd10aa5545f myap...	Off	10.0.5.0/24	-	-	251	euw2-az3 (eu-west-2b)
myapp-vpc-...	subnet-0621a8370691c8c09	Available	vpc-03aa0ebd10aa5545f myap...	Off	10.0.4.0/24	-	-	250	euw2-az2 (eu-west-2a)
myapp-vpc-...	subnet-0059dbac7366d4d2c	Available	vpc-03aa0ebd10aa5545f myap...	Off	10.0.3.0/24	-	-	247	euw2-az1 (eu-west-2c)
myapp-vpc-...	subnet-0a6865c93f2d9f1e0	Available	vpc-03aa0ebd10aa5545f myap...	Off	10.0.2.0/24	-	-	242	euw2-az3 (eu-west-2b)
myapp-vpc-...	subnet-03050240d427587b4	Available	vpc-03aa0ebd10aa5545f myap...	Off	10.0.1.0/24	-	-	242	euw2-az2 (eu-west-2a)
-	subnet-03d92ce00b3dba523	Available	vpc-03677c88f7cc2715	Off	172.31.0.0/20	-	-	4091	euw2-az1 (eu-west-2c)
-	subnet-092d85ea013bc240	Available	vpc-03677c88f7cc2715	Off	172.31.16.0...	-	-	4091	euw2-az2 (eu-west-2a)
-	subnet-051c9624093279c7e	Available	vpc-03677c88f7cc2715	Off	172.31.32.0...	-	-	4091	euw2-az3 (eu-west-2b)

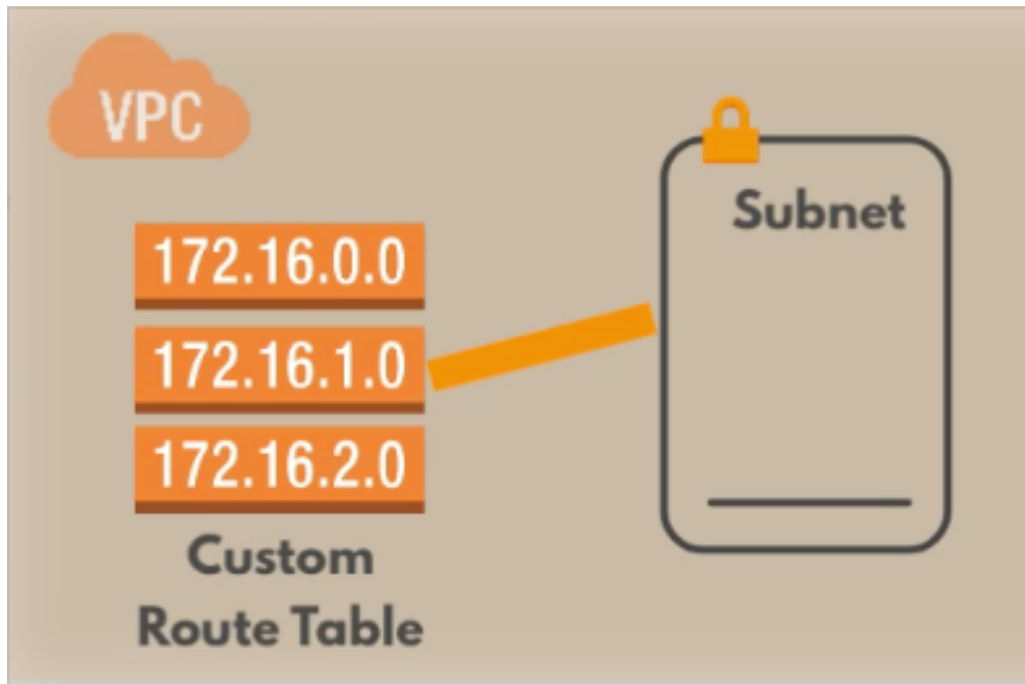
So we have 6 subnets created (the other 4 belonged to my AWS default VPC, so we can ignore them)

The screenshot shows a list of subnets. The first three subnets are private (10.0.x.x/24) and the last three are public (172.31.x.x/20). The first three subnets are associated with the VPC 'vpc-0dfb8b83df838db2c' (mya...). The last three subnets are associated with the VPC 'vpc-0371c426ad8b0bb94'.

Name	Subnet ID	State	VPC	Block Public	IPv4 CIDR	IPv6 CIDR	IPv6	Avail...	Availability Zone
vpc-0dfb8b83df838db2c mya...	subnet-0f052bce470d615a0	Available	vpc-03aa0ebd10aa5545f myap...	Off	10.0.6.0/24	-	-	251	euw2-az1 (eu-west-2c)
vpc-0dfb8b83df838db2c mya...	subnet-0b798624884354b62	Available	vpc-03aa0ebd10aa5545f myap...	Off	10.0.5.0/24	-	-	251	euw2-az3 (eu-west-2b)
vpc-0dfb8b83df838db2c mya...	subnet-0621a8370691c8c09	Available	vpc-03aa0ebd10aa5545f myap...	Off	10.0.4.0/24	-	-	250	euw2-az2 (eu-west-2a)
vpc-0dfb8b83df838db2c mya...	subnet-0059dbac7366d4d2c	Available	vpc-03aa0ebd10aa5545f myap...	Off	10.0.3.0/24	-	-	247	euw2-az1 (eu-west-2c)
vpc-0dfb8b83df838db2c mya...	subnet-0a6865c93f2d9f1e0	Available	vpc-03aa0ebd10aa5545f myap...	Off	10.0.2.0/24	-	-	242	euw2-az3 (eu-west-2b)
vpc-0dfb8b83df838db2c mya...	subnet-03050240d427587b4	Available	vpc-03aa0ebd10aa5545f myap...	Off	10.0.1.0/24	-	-	242	euw2-az2 (eu-west-2a)
vpc-0371c426ad8b0bb94	subnet-03d92ce00b3dba523	Available	vpc-03677c88f7cc2715	Off	172.31.0.0/20	-	-	4091	euw2-az1 (eu-west-2c)
vpc-0371c426ad8b0bb94	subnet-092d85ea013bc240	Available	vpc-03677c88f7cc2715	Off	172.31.16.0/20	-	-	4091	euw2-az2 (eu-west-2a)
vpc-0371c426ad8b0bb94	subnet-051c9624093279c7e	Available	vpc-03677c88f7cc2715	Off	172.31.32.0/20	-	-	4091	euw2-az3 (eu-west-2b)

3 private subnets and 3 public subnets.

Let's check the route table association (see notes in Module 12 - chapter 12 subnet association)



Available	vpc-0d1b8b83df838db2c mya...	Off	10.0.3.0/24
3 private Subnets => NAT Gateway			
Available	vpc-0d1b8b83df838db2c mya...	Off	10.0.1.0/24
3 public Subnets => Internet Gateway			

Private subnet -> NAT

myapp-vpc-private-eu-west-2c is associated with the route table

rtb-0d5df904fb3cf5c40 / myapp-vpc-private

Subnets (1/9) [Info](#)

Find subnets by attribute or tag

	Name	Subnet ID	State	VPC	Block Publi...	IPv4 C
☐	myapp-vpc-public-eu-west-2c	subnet-0dec9610ac3db0224	Available	vpc-0b4e83c353f2cb6c6 myapp...	Off	10.0.6
☐	myapp-vpc-public-eu-west-2b	subnet-0e51ed9fb91a95164	Available	vpc-0b4e83c353f2cb6c6 myapp...	Off	10.0.5
☐	myapp-vpc-public-eu-west-2a	subnet-07e1c4005375a04c8	Available	vpc-0b4e83c353f2cb6c6 myapp...	Off	10.0.4
☒	myapp-vpc-private-eu-west-2c	subnet-077810120a66c911f	Available	vpc-0b4e83c353f2cb6c6 myapp...	Off	10.0.3
☐	myapp-vpc-private-eu-west-2b	subnet-0b799030b810ff3c3	Available	vpc-0b4e83c353f2cb6c6 myapp...	Off	10.0.2
☐	myapp-vpc-private-eu-west-2a	subnet-0f01eb190d67cd7cd	Available	vpc-0b4e83c353f2cb6c6 myapp...	Off	10.0.1
☐	-	subnet-03da2ce00b3dba523	Available	vpc-03677c88f7cc2f715	Off	172.3'
☐	-	subnet-092c85ea013cbc240	Available	vpc-03677c88f7cc2f715	Off	172.3'
☐	-	subnet-051c96240932f9c7e	Available	vpc-03677c88f7cc2f715	Off	172.3'

subnet-077810120a66c911f / myapp-vpc-private-eu-west-2c

Details | Flow logs | **Route table** | Network ACL | CIDR reservations | Sharing | Tags

Route table: [rtb-0d5df904fb3cf5c40 / myapp-vpc-private](#)

Routes (2)

Filter routes

Destination	Target
10.0.0.0/16	local
0.0.0.0/0	nat-0b92fa6879c26d83d

Here is the information of the route table [rtb-0d5df904fb3cf5c40](#)

Route tables (1/4) [Info](#)

Find route tables by attribute or tag

	Name	Route table ID	Explicit subnet associ...	Edge a...	Main	VPC	Owner ID
☒	myapp-vpc-private	rtb-0d5df904fb3cf5c40	3 subnets	-	No	vpc-0b4e83c353f2cb6c6 myapp-vpc	706976333940
☐	myapp-vpc-default	rtb-092316f096d3709b1	-	-	Yes	vpc-0b4e83c353f2cb6c6 myapp-vpc	706976333940
☐	myapp-vpc-public	rtb-06a9b5c5a8fe9dd9a	3 subnets	-	No	vpc-0b4e83c353f2cb6c6 myapp-vpc	706976333940
☐	-	rtb-0118aa5c82fb9ff	-	-	Yes	vpc-03677c88f7cc2f715	706976333940

rtb-0d5df904fb3cf5c40 / myapp-vpc-private

Details | Routes | **Subnet associations** | Edge associations | Route propagation | Tags

Explicit subnet associations (3)

Find subnet association

Name	Subnet ID	IPv4 CIDR	IPv6 CIDR
myapp-vpc-private-eu-west-2a	subnet-0f01eb190d67cd7cd	10.0.1.0/24	-
myapp-vpc-private-eu-west-2c	subnet-077810120a66c911f	10.0.3.0/24	-
myapp-vpc-private-eu-west-2b	subnet-0b799030b810ff3c3	10.0.2.0/24	-

Subnets without explicit associations (0)

The following subnets have not been explicitly associated with any route tables and are therefore associated with the main route table:

Find subnet association

Name	Subnet ID	IPv4 CIDR	IPv6 CIDR
------	-----------	-----------	-----------

No subnets without explicit associations

And we can see that 3 private subnets are associated with this route table.

So if we check private subnet for AZ 2b it is associated with the route table above as well (same route table id -> [rtb-0d5df904fb3cf5c40 / myapp-vpc-private](#))

subnet-0b799030b810ff3c3 / myapp-vpc-private-eu-west-2b	
Details	Flow logs
Route table	Network ACL
CIDR reservations	Sharing
Tags	
Route table: rtb-0d5df904fb3cf5c40 / myapp-vpc-private	
Routes (2)	
Filter routes	
Destination	Target
10.0.0.0/16	local
0.0.0.0/0	nat-0b92fa6879c26d83d

And same for private subnet in AZ 2a

subnet-0f01eb190d67cd7cd / myapp-vpc-private-eu-west-2a	
Details	Flow logs
Route table	Network ACL
CIDR reservations	Sharing
Tags	
Route table: rtb-0d5df904fb3cf5c40 / myapp-vpc-private	
Routes (2)	
Filter routes	
Destination	Target
10.0.0.0/16	local
0.0.0.0/0	nat-0b92fa6879c26d83d

Public subnet -> IGW

myapp-vpc-public-eu-west-2c Is associated with route table

rtb-06a9b5c5aefe9dd9a / myapp-vpc-public

Subnets (1/9)	
Find subnets by attribute or tag	
Name	Subnet ID
State	VPC
Block Publ...	
myapp-vpc-public-eu-west-2c	subnet-0dec9610ac3db0224
myapp-vpc-public-eu-west-2b	subnet-0e51ed9fb91a95164
myapp-vpc-public-eu-west-2a	subnet-07e1c4005375a04c8
myapp-vpc-private-eu-west-2c	subnet-077810120a66c911f
myapp-vpc-private-eu-west-2b	subnet-0b799030b810ff3c3
myapp-vpc-private-eu-west-2a	subnet-0f01eb190d67cd7cd
-	subnet-03da2ce00b3dba523
-	subnet-092c85ea013cb240
-	subnet-051c96240932f9c7e
subnet-0dec9610ac3db0224 / myapp-vpc-public-eu-west-2c	
Details	Flow logs
Route table	Network ACL
CIDR reservations	Sharing
Tags	
Route table: rtb-06a9b5c5aefe9dd9a / myapp-vpc-public	
Routes (2)	
Filter routes	
Destination	Target
10.0.0.0/16	local
0.0.0.0/0	lgw-00959e38045b4928c

Here is the information of the route table rtb-06a9b5c5aefe9dd9a / myapp-vpc-public

VPC dashboard < AWS Global View  Filter by VPC

Virtual private cloud

- Your VPCs
- Subnets
- Route tables
- Internet gateways
- Egress-only internet gateways
- Carrier gateways
- DHCP option sets
- Elastic IPs
- Managed prefix lists
- NAT gateways
- Peering connections
- Route servers

Security

- Network ACLs
- Security groups

PrivateLink and Lattice

- Getting started
- Endpoints
- Endpoint services
- Service networks
- Lattice services

Route tables (1/4) info Last updated 5 minutes ago

Find route tables by attribute or tag

Name	Route table ID	Explicit subnet associ...	Edge a...	Main	VPC	Owner ID
☐ myapp-vpc-private	rtb-0d5df904fb3cfc540	3 subnets	-	No	vpc-0b4e83c353f2cb6c6 myapp-vpc	70697633
☐ myapp-vpc-default	rtb-092316f09d37f06b1	-	-	Yes	vpc-0b4e83c353f2cb6c6 myapp-vpc	70697633
☒ myapp-vpc-public	rtb-06a9b5c5aefe9dd9a	3 subnets	-	No	vpc-0b4e83c353f2cb6c6 myapp-vpc	70697633
☐ -	rtb-0118aea5c82fb9ff	-	-	Yes	vpc-03677c88f7cc2f715	70697633

rtb-06a9b5c5aefe9dd9a / myapp-vpc-public

Details | Routes | **Subnet associations** | Edge associations | Route propagation | Tags

Explicit subnet associations (3)

Find subnet association

Name	Subnet ID	IPV4 CIDR	IPV6 CIDR
myapp-vpc-public-eu-west-2c	subnet-0dec9610ac3db0224	10.0.6.0/24	-
myapp-vpc-public-eu-west-2a	subnet-07e1c4005375a04c8	10.0.4.0/24	-
myapp-vpc-public-eu-west-2b	subnet-0e51ed9fb91a95164	10.0.5.0/24	-

And we can see that 3 public subnets are associated with this route table.

So if we check public subnet for AZ 2b it is associated with the route table above as well (same route table id -> **rtb-06a9b5c5aefe9dd9a / myapp-vpc-public**)

subnet-0e51ed9fb91a95164 / myapp-vpc-public-eu-west-2b

Details | Flow logs | **Route table** | Network ACL | CIDR reservations | Sharing | Tags

Route table: [rtb-06a9b5c5aefe9dd9a / myapp-vpc-public](#)

Routes (2)

Filter routes

Destination	Target
10.0.0.0/16	local
0.0.0.0/0	igw-00959e38045b4928c

And same for public subnet in AZ 2a

subnet-07e1c4005375a04c8 / myapp-vpc-public-eu-west-2a

Details | Flow logs | **Route table** | Network ACL | CIDR reservations | Sharing | Tags

Route table: [rtb-06a9b5c5aefe9dd9a / myapp-vpc-public](#)

Routes (2)

Filter routes

Destination	Target
10.0.0.0/16	local
0.0.0.0/0	igw-00959e38045b4928c

Now let's check the security groups

Security Groups (7) Info						
Find security groups by attribute or tag						
☐	Name	Security group ID	Security group name	VPC ID	Description	
☐	myapp-vpc-default	sg-06f660018c173b72c	default	vpc-03aa0ebd10aa5545f	default VPC security group	
☐	myapp-eks-cluster-node	sg-00fbfa25749353734	myapp-eks-cluster-node-20260624145...	vpc-03aa0ebd10aa5545f	EKS node shared security group	
☐	myapp-eks-cluster-cluster	sg-03bf4d01a4515c434	myapp-eks-cluster-cluster-2026062414...	vpc-03aa0ebd10aa5545f	EKS cluster security group	
☐	eks-cluster-sg-myapp-eks-cluster-1899522018	sg-0171a9b3dfb871388	eks-cluster-sg-myapp-eks-cluster-1899...	vpc-03aa0ebd10aa5545f	EKS created security group applied to ENI that is	
☐	-	sg-0638fd0a242d48af6	default	vpc-03677c887cc2f715	default VPC security group	
☐	-	sg-087818984cfa47e94	my-sg	vpc-03677c887cc2f715	My SG	
☐	-	sg-09e272e3b7cc7f80b	security-group-docker-server	vpc-03677c887cc2f715	launch-wizard-1 created 2026-03-24T14:19:13...	

Four sg have been created by the VPC module

- Default -> we aren't using it
- Eks-cluster-sg-myapp-eks-cluster -> And this sg allows all traffic on any port within the cluster.

Security Groups (1/7) [Info](#)
Actions
Export security groups to CSV
Create security group

Find security groups by attribute or tag

☐	Name	Security group ID	Security group name	VPC ID	Description
☐	myapp-vpc-default	sg-06f660018c173b72c	default	vpc-03aa0ebd10aa5545f	default VPC security group
☐	myapp-eks-cluster-node	sg-00fbfa25749353734	myapp-eks-cluster-node-20260624145...	vpc-03aa0ebd10aa5545f	EKS node shared security group
☐	myapp-eks-cluster-cluster	sg-03bf4d01a4515c434	myapp-eks-cluster-cluster-2026062414...	vpc-03aa0ebd10aa5545f	EKS cluster security group
☒	eks-cluster-sg-myapp-eks-cluster-1899522018	sg-0171a9b3dfb871388	eks-cluster-sg-myapp-eks-cluster-1899...	vpc-03aa0ebd10aa5545f	EKS created security group applied to ENI that
☐	-	sg-0638fd0a242d48af6	default	vpc-03677c887cc2f715	default VPC security group
☐	-	sg-087818984cfa47e94	my-sg	vpc-03677c887cc2f715	My SG
☐	-	sg-09e272e3b7cc7f80b	security-group-docker-server	vpc-03677c887cc2f715	launch-wizard-1 created 2026-03-24T14:19:13...

sg-0171a9b3dfb871388 - eks-cluster-sg-myapp-eks-cluster-1899522018

Details
Inbound rules
Outbound rules
Sharing
VPC associations
Related resources - new
Tags

Inbound rules (1/1)

Manage tags Edit inbound rules

Find inbound rules by attribute or tag

☒	Name	Security group rule ID	IP version	Type	Protocol	Port range	Source
☒	-	sg-0d5f481bfda1fc8d4	-	All traffic	All	All	sg-0171a9b3dfb871388 / eks-clus...

— tangent —

Q:

I have this sg eks-cluster-sg-myapp-eks-cluster-1899522018 and its description is "EKS created security group applied to ENI that is attached to EKS Control Plane master nodes, as well as any managed workloads." Can you explain?

A:

This is the EKS cluster security group that AWS automatically creates when you provision an EKS cluster — it's not something you define in Terraform, it's created by AWS itself regardless of which modules you use.

Why it exists

EKS needs a shared security group that is attached to:

- The EKS control plane ENIs (the elastic network interfaces AWS provisions in your VPC so the control plane can communicate with your nodes)

- Managed node groups (EC2 worker nodes) and Fargate pods

This lets the control plane and workers talk to each other without you having to manually manage that plumbing.

Why the rules look the way they do

Inbound: All traffic from itself (sg-0928cc8771936789b, which is the same SG).

- This is self-referencing — any resource in this SG can reach any other resource in this SG on any port. This is how the control plane ENIs and worker nodes communicate freely with each other.
- The description mentioning EFA (Elastic Fabric Adapter) is just AWS boilerplate for self-referencing SG rules; EFA isn't necessarily in use.

Outbound rule 1: All traffic to itself (same self-reference, same reason).

Outbound rule 2: All traffic to 0.0.0.0/0.

- This allows worker nodes to reach the internet — necessary for pulling container images, calling AWS APIs (ECR, S3, SSM, etc.), and general egress. If you're in a private subnet with a NAT gateway, traffic still routes through that — this rule just doesn't restrict the destination.

In short

AWS created it automatically to wire up control-plane-to-node communication. It's expected, correct, and you should leave it alone. The eks module in Terraform does register it (you can see it referenced as `cluster_security_group_id` in the module outputs), but the SG itself is owned and managed by AWS EKS, not by your Terraform state.

Q:

what do you mean by "If you're in a private subnet with a NAT gateway, traffic still routes through that — this rule just doesn't restrict the destination."?

A:

The 0.0.0.0/0 outbound rule on a security group means "allow traffic to any IP address." But a security group only controls whether traffic is allowed — it doesn't control how traffic is routed. Routing is handled separately by your VPC route tables.

So these two things are independent:

- Security group rule: "you are allowed to send traffic anywhere"
- Route table: "here is the actual path that traffic takes to get there"

If your worker nodes are in a private subnet, the route table for that subnet has

no direct internet gateway route. Instead it has a route like 0.0.0.0/0 → nat-xxxxxxx. So even though the security group permits outbound traffic to any destination, all that traffic is physically forced through the NAT gateway by the route table — the node never gets a direct internet connection.

Contrast that with a public subnet, where the route table points 0.0.0.0/0 at an internet gateway directly, so nodes with public IPs can reach the internet without going through NAT.

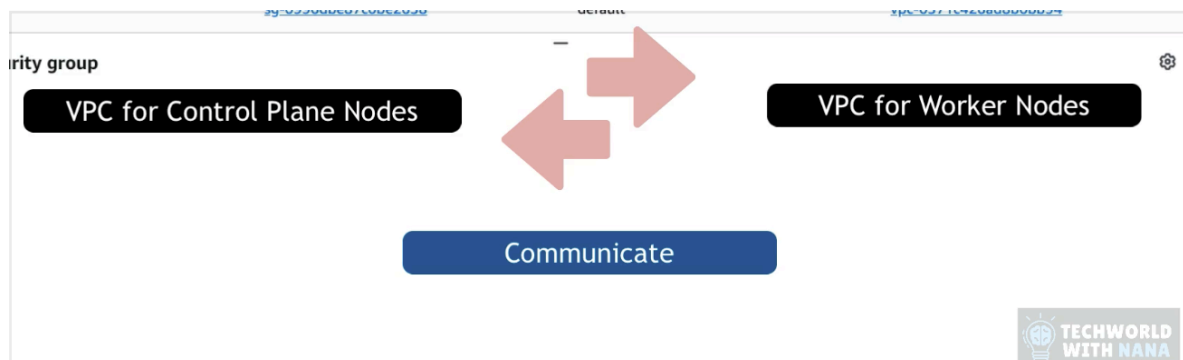
The short version: the 0.0.0.0/0 outbound rule isn't a security concern in a private subnet because the route table is the actual gatekeeper for where traffic can go.

—end of tangent —

Then we have the sg for the node group and the cluster

- myapp-eks-cluster-node
- myapp-eks-cluster-cluster

So we have two VPCs



With these two sg we are opening the ports of the services so those services can talk to each other between those VPCs

And the SG are giving the minimum required permissions to define who can access those ports that we opened



The security groups are using

- In module 12 - chapter 12 we created a security group and assigned IP addresses in the cidr_blocks

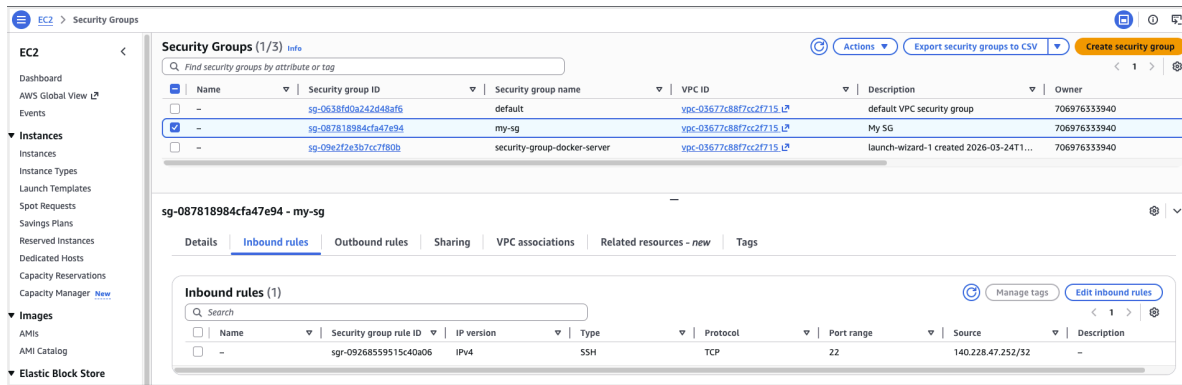
```
63 resource "aws_security_group" "myapp-sg" {
64     name = "myapp-sg"
65     vpc_id = aws_vpc.myapp-vpc.id
66
67     ingress {
68         from_port = 22
69         to_port = 22
70         protocol = "TCP"
71         cidr_blocks = [var.my_ip]
72     }
73
74     ingress {
75         from_port = 8080
76         to_port = 8080
77         protocol = "TCP"
78         cidr_blocks = ["0.0.0.0/0"]
79     }
80 }
```

Another ingress example:

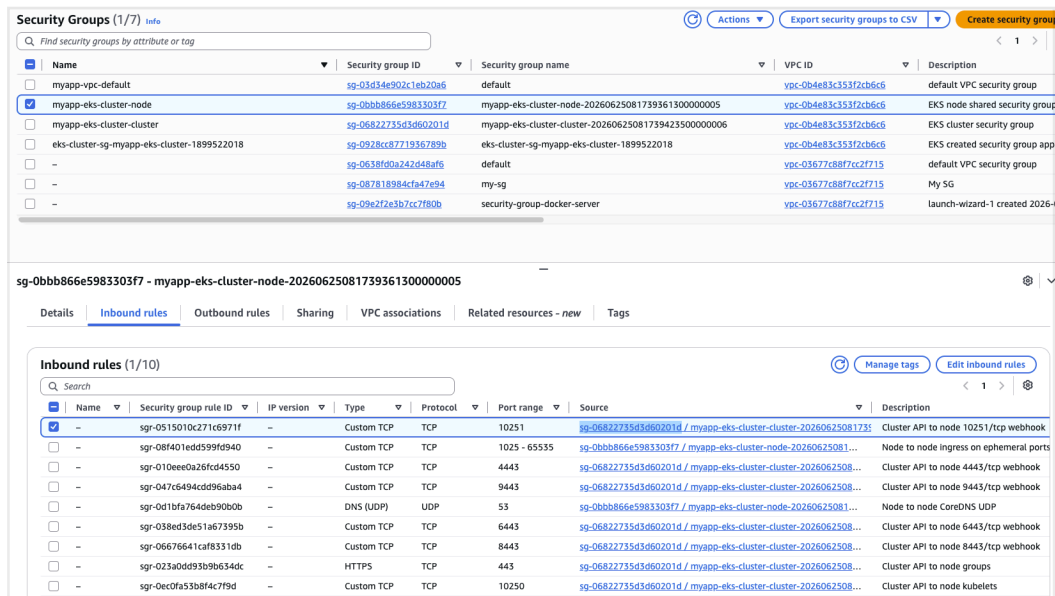
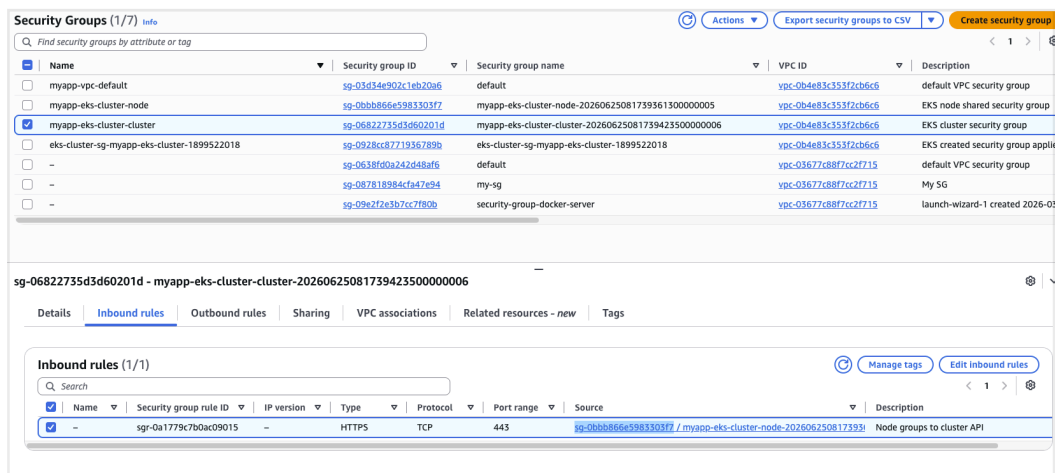
```
ingress {
  from_port = 443
  to_port   = 443
  protocol  = "tcp"
  cidr_blocks = ["10.0.0.0/16"]
}
```

That cidr_blocks is the IP range source. You're saying "allow port 443 from any IP in the range 10.0.0.0/16."

- And In module 9 - chapter 12 I created a security group and the source column shows an IP address



But now with terraform our sources are not IP addresses, instead they are security groups



EKS module does:

```
ingress {
  from_port    = 443
  to_port      = 443
  protocol     = "tcp"
```

```
source_security_group_id = aws_security_group.node.id
}
```

Here there's no IP at all — the source is another security group.

A fundamental **AWS security concept** -> using security groups as sources instead of IP ranges.

IP range as source (less ideal)

You allow traffic from a specific IP or CIDR block, e.g. 10.0.0.0/16. The problem is that IPs can change — nodes get replaced, autoscaling brings up new instances with different IPs, pods shift around. You'd have to constantly update rules to keep up, and you might accidentally allow IPs you didn't intend to.

Security group as source (what Terraform does)

Instead of saying "allow traffic from this IP", you say "allow traffic from anything that has this security group attached." For example:

- Node SG allows port 10250 from the cluster SG
- This means: any resource wearing the cluster SG badge can reach port 10250 on nodes
- It doesn't matter what IP the control plane ENI has — if it has that SG attached, it gets in

Why this is better for EKS specifically

EKS is highly dynamic:

- Worker nodes are constantly being replaced (upgrades, autoscaling, spot interruptions)
- Control plane ENIs have IPs managed by AWS that you don't control
- Pods move between nodes

With IP-based rules you'd be chasing IPs forever. With SG-based rules, you're granting access based on identity (what role does this resource play) rather than location (what IP does it have right now). A new node automatically gets access the moment the node SG is attached to it — no rule changes needed.

This is exactly what you saw in your rules — every source was another SG, never a CIDR block.

The EKS Terraform module deliberately uses `source_security_group_id` everywhere instead of `cidr_blocks`.

Now let's understand each security group inbound and outbound rules created by terraform EKS module

- myapp-eks-cluster-cluster — inbound only, one rule

Port	From	Why
443	node SG	Worker nodes calling the Kubernetes API server (kubectl, kubelets, pods using in-cluster config)

No outbound rules needed here because the AWS-managed eks-cluster-sg-* (which is also attached to the control plane ENIs) already has the 0.0.0.0/0 outbound rule. AWS combines rules across all SGs on an ENI, so the control plane can still send traffic out.

- myapp-eks-cluster-node — inbound rules

Here we can see how the sg myapp-eks-cluster-node has the security groups as sources

Name

☐

myapp-vpc-default

☒

myapp-eks-cluster-node

☐

myapp-eks-cluster-cluster

☐

eks-cluster-sg-myapp-eks-cluster-1899522018

▼

Security group ID

sg-03d34e902c1eb20a6

sg-0bbb866e5983303f7

sg-06822735d3d60201d

sg-0928cc8771936789b

▼

Security group name

default

myapp-eks-cluster-node-20260625081739361300000005

myapp-eks-cluster-cluster-20260625081739423500000006

eks-cluster-sg-myapp-eks-cluster-1899522018

▼

VPC ID

vpc-0b4e83c35372cb6c6

vpc-0b4e83c35372cb6c6

vpc-0b4e83c35372cb6c6

vpc-0b4e83c35372cb6c6

▼

Description

default VPC security group

EKS node shared security group

EKS cluster security group

EKS created security group app

sg-0bbb866e5983303f7 - myapp-eks-cluster-node-20260625081739361300000005

Control plane → nodes (source: cluster SG -> sg-06822735d3d60201d) 🙌

Port	Why
10250	Kubelet API - control plane uses this for kubectl exec, kubectl logs, port-forward
443, 4443, 6443, 8443, 9443, 10251	Admission webhook ports - when you install things like AWS Load Balancer Controller, Karpenter, cert-manager etc., they run webhook servers on nodes; the API server calls back to these ports to validate/mutate resources

myapp-vc-derault	sg-030349e90c1e0c080	derault	ypc-004e83c3231cc08c0	derault VPC security group
myapp-eks-cluster-node	sg-0bbb866e5983303f7	myapp-eks-cluster-node-20260625081739361300000005	ypc-0b4e83c353f2cb6c6	EKS node shared security group
myapp-eks-cluster-cluster	sg-06822735d3d60201d	myapp-eks-cluster-cluster-20260625081739423500000006	ypc-0b4e83c353f2cb6c6	EKS cluster security group
eks-cluster-sg-myapp-eks-cluster-1899522018	sg-0928cc8771936789b	eks-cluster-sg-myapp-eks-cluster-1899522018	ypc-0b4e83c353f2cb6c6	EKS created security group app
-	sg-0638f0a242d48af5	default	ypc-03677c887cc2f715	default VPC security group
-	sg-087818984cf47e94	my-sg	ypc-03677c887cc2f715	My SG
-	sg-09e2f2e3b7cc7f80b	security-group-docker-server	ypc-03677c887cc2f715	launch-wizard-1 created 2026-

sg-0bbb866e5983303f7 - myapp-eks-cluster-node-20260625081739361300000005				
Details	Inbound rules	Outbound rules	Sharing	VPC associations

Inbound rules (10)				
Security group rule ID	IP version	Type	Protocol	Port range
sg-0515010c271c6971f	-	Custom TCP	TCP	10251
sg-010ee0a2e6fcd4550	-	Custom TCP	TCP	4443
sg-047c6494cd96aba4	-	Custom TCP	TCP	9443
sg-038ed5de51a67395b	-	Custom TCP	TCP	6443
sg-06678641caf8331db	-	Custom TCP	TCP	8443
sg-023a0dd9399b634dc	-	HTTP	TCP	443
sg-0ec0fa53b8f4c7f9d	-	Custom TCP	TCP	10250
sg-08f401edd599fd940	-	Custom TCP	TCP	1025 - 65535
sg-0d1bfa764deb90b0b	-	DNS (UDP)	UDP	53
sg-090c320ce8e7193f4	-	DNS (TCP)	TCP	53

Node → node (source: node SG, self-referencing -> [sg-0bbb866e5983303f7](#))
👉

Port	Why
1025–65535	Ephemeral ports – return traffic for pod-to-pod and node-to-node TCP connections
53 UDP + 53 TCP	CoreDNS – all pods do DNS lookups against CoreDNS pods running on nodes

Outbound: 0.0.0.0/0

Nodes need unrestricted egress to pull images from ECR, call AWS APIs, reach the API server, etc.

Another distinction regarding the TF created security groups ⭐

These are both created by the Terraform EKS module and serve distinct purposes.

myapp-eks-cluster-cluster

This is the additional cluster security group that Terraform manages for the control plane. It's separate from the AWS-managed one we discussed earlier (eks-cluster-sg-*). Your EKS control plane ends up with both attached.

This one is where the module adds rules for things like allowing your kubectl access — for example, if you configured cluster_endpoint_public_access or specified cluster_additional_security_group_ids in your Terraform, rules land here. It gives you a Terraform-managed SG to attach your own rules to without touching the AWS-owned one.

myapp-eks-cluster-node

This is the worker node security group attached to all nodes in your managed node groups. Its typical rules are:

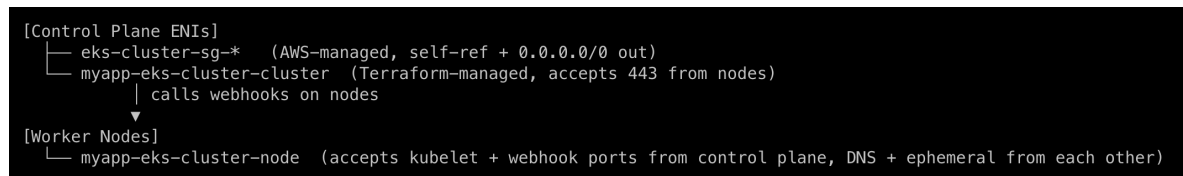
- Inbound from the cluster SGs — so the control plane can reach kubelet (port 10250) on your nodes for health checks and exec/logs
- Inbound self-reference — node-to-node communication (needed for pod-to-pod traffic across nodes)
- Outbound 0.0.0.0/0 — so nodes can pull images from ECR, call AWS APIs, etc.

How the three security groups relate together?

All three work together:

SG	Owner	Attached to
<code>eks-cluster-sg-*</code>	AWS	Control plane ENIs + nodes
<code>myapp-eks-cluster-cluster</code>	Terraform	Control plane
<code>myapp-eks-cluster-node</code>	Terraform	Worker nodes

The picture as a whole



— tangent —

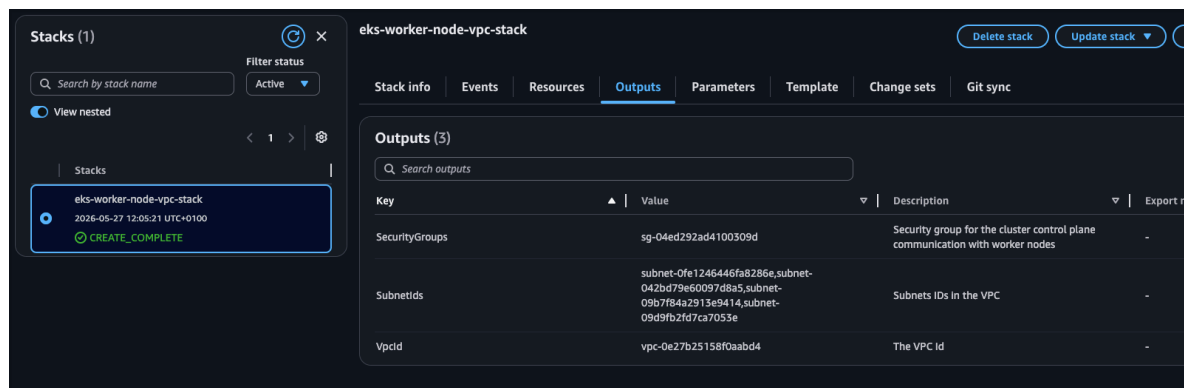
Q:

you said "This is the EKS cluster security group that AWS automatically creates when you provision an EKS cluster — it's not something you define in Terraform, it's created by AWS itself regardless of which modules you use."

(see Module 11 - chapter 2) I remember creating an EKS cluster manually but first I created a VPC for my worker nodes and when the VPC was created I got 3 outputs one of which was a security group I have this note:

"This VPC with k8s config created a security group 'sg-04ed292ad4100309d' which is the communication endpoint — it allows traffic between your worker nodes and the AWS managed control plane."

and this screenshot



is this the sg that you were mentioning? if so it was created when provisioning the VPC for the worker nodes and then used to create the eks cluster?

A:

No, these are two different things from two different cluster setups — you're comparing your old manual cluster (CloudFormation) with your new Terraform cluster.

sg-04ed292ad4100309d (CloudFormation, manual cluster)

- You defined this yourself in your CloudFormation template as part of the VPC stack
- You then passed it explicitly to EKS when creating the cluster manually
- It served the same purpose — control plane to worker node communication — but it was user-defined, not auto-created by AWS

eks-cluster-sg-myapp-eks-cluster-1899522018 (Terraform cluster, current)

- This is what I was referring to as auto-created by AWS
- AWS EKS creates this automatically every time a cluster is provisioned, regardless of how (console, CLI, Terraform, CloudFormation)
- You don't define it anywhere — it just appears

The key distinction is that when you created your cluster manually, you had to create that control-plane-to-node SG yourself in CloudFormation and wire it up. When you use Terraform's EKS module, AWS still auto-creates its own cluster SG, and Terraform also creates the additional myapp-eks-cluster-cluster and myapp-eks-cluster-node SGs — so you end up with three SGs handling what you previously did with one manual one.

Same concept, different execution — the Terraform module just handles more of the plumbing for you.

Q:

this is confusing me " AWS EKS creates this automatically every time a cluster is provisioned, regardless of how (console, CLI, Terraform, CloudFormation)" that means that when I manually created the EKS I got a sg as well? or I could have got it but because I manually wired the VPC's sg to the EKS then EKS didn't create this automatic sg?

A:

Yes, AWS EKS always creates that cluster SG automatically — you got one when you created the manual cluster too. It's unavoidable.

What differs is what you do with it. In your manual setup, you likely didn't use it — you explicitly passed sg-04ed292ad4100309d from your CloudFormation VPC stack to the EKS cluster configuration to handle control plane to worker node communication. That SG did the job, and you probably never noticed the auto-created one sitting there alongside it.

With Terraform, you're not manually wiring anything — the EKS module handles it all, so the auto-created SG is actively used as part of the setup.

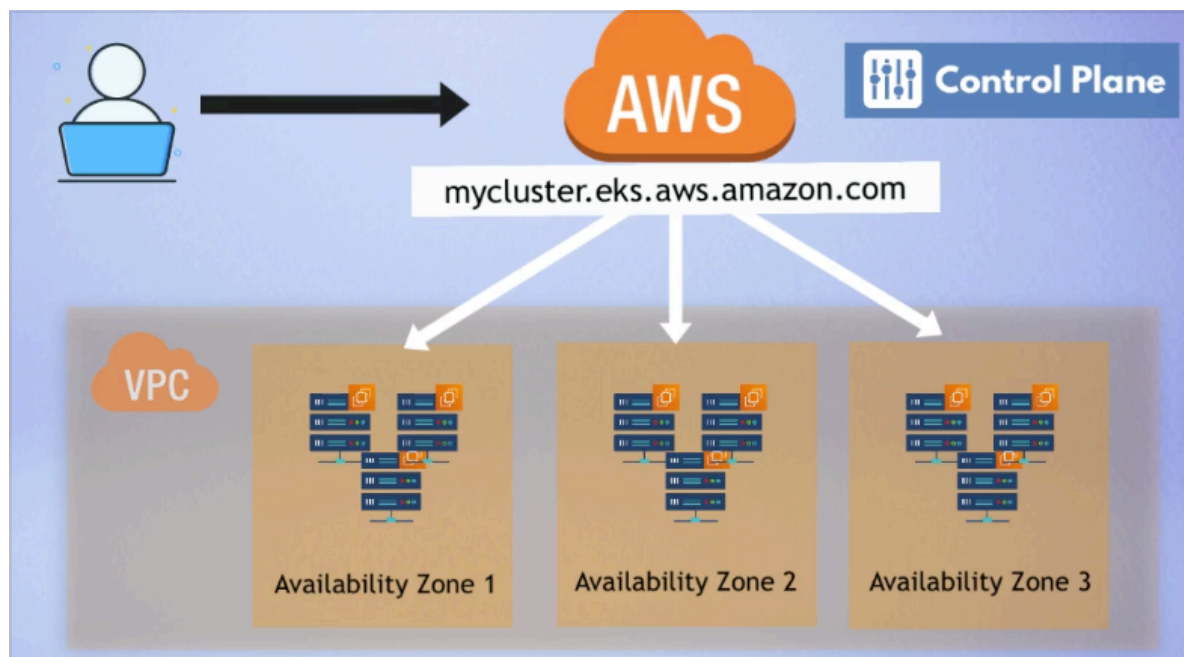
So to directly answer your question: EKS didn't skip creating the automatic SG when you did it manually. It created it, you just didn't use it. You brought your own SG instead.

— end of tangent —

If we are debugging a problem with connectivity we come here to the security group and check 💡

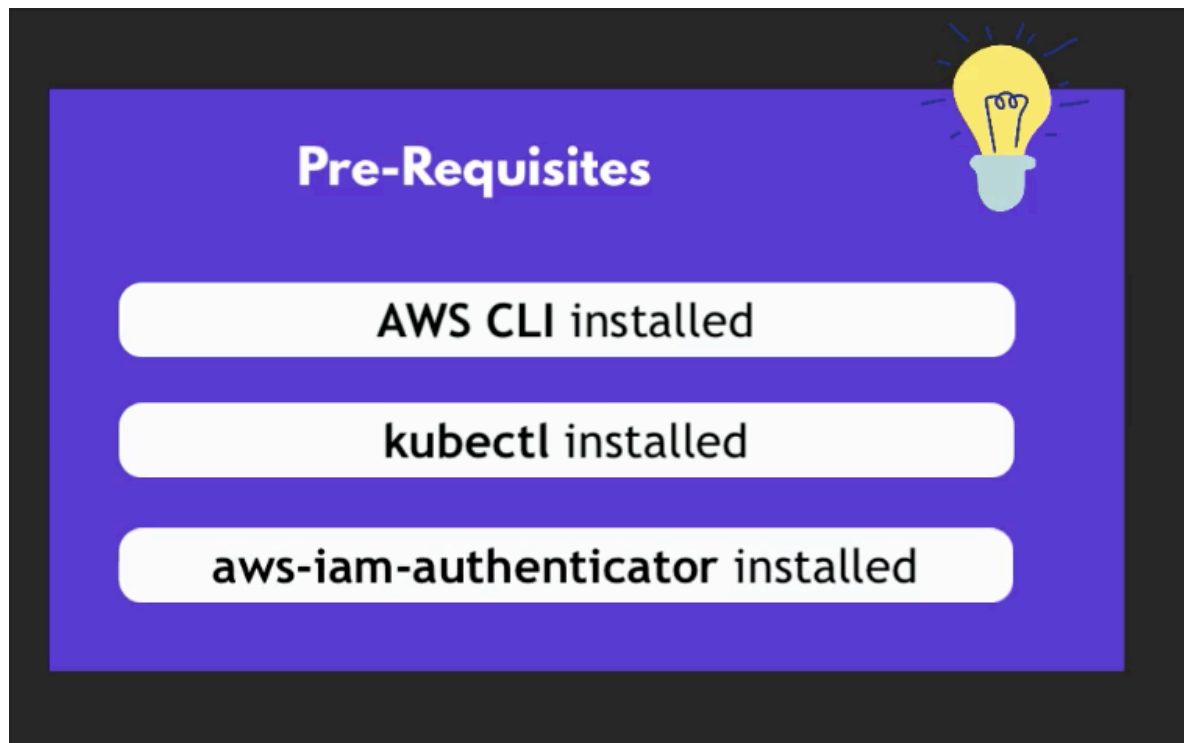
Deploy nginx-app into our cluster

How do we connect to our cluster using kubectl?



1. Configure our environment so kubectl can connect to our cluster
-> `aws eks update-kubeconfig --name myapp-eks-cluster --region eu-west-2`

The above will update our kubeconfig file, so it has the correct cacert (ca certificate), token, IAM authenticator, etc)



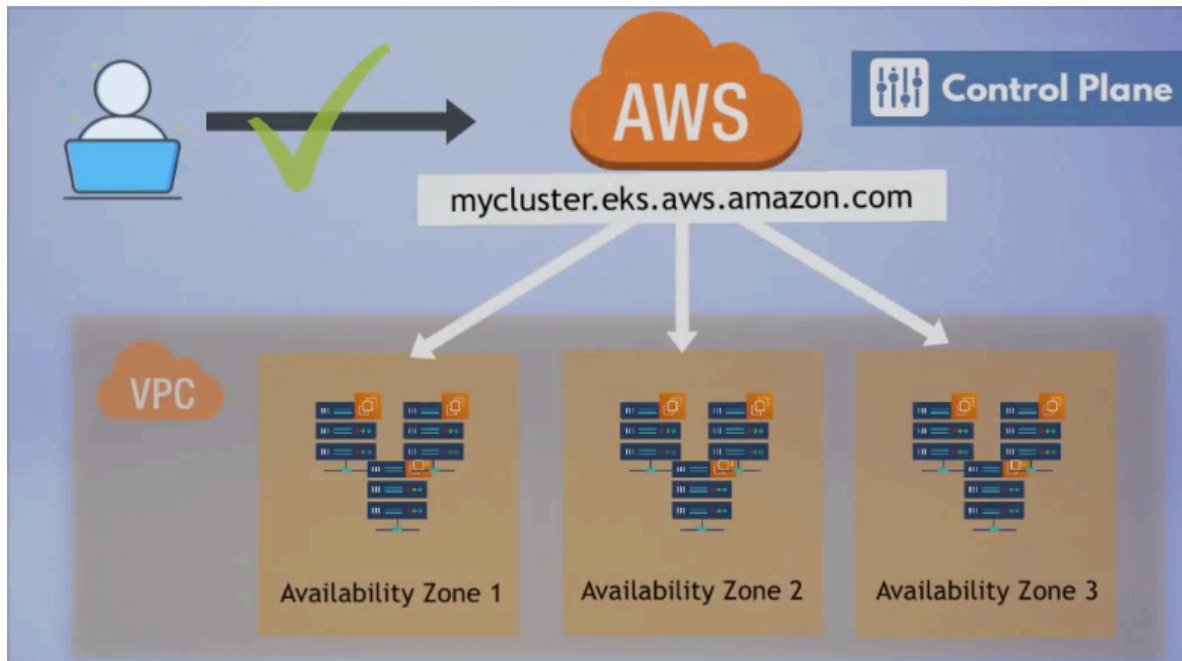
```
> aws eks update-kubeconfig --name myapp-eks-cluster --region eu-west-2
Added new context arn:aws:eks:eu-west-2:706976333940:cluster/myapp-eks-cluster to /Users/astriid/.kube/config
```

And we can see that the context was configured in /Users/astriid/.kube/config

So now kubectl and iam authenticator should be able to get all the info they need to authenticate with AWS and then Auth with the EKS cluster.

-> kubectl get node

```
> kubectl get node
NAME                                STATUS    ROLES    AGE    VERSION
ip-10-0-1-144.eu-west-2.compute.internal Ready    <none>   169m   v1.36.2-eks-93b80c6
ip-10-0-2-44.eu-west-2.compute.internal Ready    <none>   169m   v1.36.2-eks-93b80c6
ip-10-0-3-243.eu-west-2.compute.internal Ready    <none>   169m   v1.36.2-eks-93b80c6
```



We can see our 3 worker nodes that we deployed 🤔

We have not deployed any app just yet so we don't have pods

-> kubectl get pod

```
> kubectl get pod
No resources found in default namespace.
```

To test we can deploy a simple nginx app from the terminal (to get the nginx-config.yaml file I found it in my GitHub repo https://github.com/AstridCaballero/Module_11_Kubernetes_AWS_EKS/blob/Module_11/EKS_Autoscaler_chapter-3/nginx-config.yaml and then I copied and paste it in my current working branch)

-> kubectl apply -f terraform/nginx-config.yaml

```
> kubectl apply -f terraform/nginx-config.yaml
deployment.apps/nginx created
service/nginx created
```

-> kubectl get pod

```
> kubectl get pod
NAME                                READY   STATUS    RESTARTS   AGE
nginx-5cf8dc6bc5-sgfs9             1/1     Running   0           2m31s
```

-> kubectl get svc

```
> kubectl get svc
NAME      TYPE          CLUSTER-IP      EXTERNAL-IP      PORT(S)
kubernetes ClusterIP      172.20.0.1      <none>           443/TCP
nginx     LoadBalancer  172.20.246.247  af48688251f224c48be71ffe585aeee9-576210719.eu-west-2.elb.amazonaws.com 80:31707/TCP
```

And the service is a loadbalancer so it is accessible externally

-> af48688251f224c48be71ffe585aeee9-576210719.eu-west-2.elb.amazonaws.com

And if we check AWS UI, we can see there is one loadbalancer

EC2
[Dashboard](#)
[AWS Global View](#)
[Events](#)
Instances
[Instances](#)
[Instance Types](#)
[Launch Templates](#)
[Spot Requests](#)
[Savings Plans](#)

Resources
You are using the following Amazon EC2 resources in the Europe (London) Region:

Instances (running)	3	Auto Scaling Groups	1	Capacity Reservations
Dedicated Hosts	0	Elastic IPs	1	Instances
Key pairs	2	Load balancers	1	Placement groups
Security groups	8	Snapshots	0	Volumes

And we can see that is available in the 3 AZs -> accessible and highly available

Load balancers (1/1)
Elastic Load Balancing scales your load balancer capacity automatically in response to changes in incoming traffic.

Name	State	Type	Scheme	IP address type	VPC ID	Availability Zones	Security
af48688251f224c48be...	-	classic	-	-	vpc-0b4e83c353f2cb6c6	3 Availability Zones	sg-0bd7...

Load balancer: af48688251f224c48be71ffe585aeee9
[Details](#) [Listeners](#) [Network mapping](#) [Security](#) [Health checks](#) [Target instances](#) [Monitoring](#) [Attributes](#) [Tags](#)

Details

Load balancer type Classic	Status 1 of 3 instances in service	VPC vpc-0b4e83c353f2cb6c6	Date created June 25, 2026, 12:00
Scheme Internet-facing	Hosted zone ZHURV8PSTC4K8	Availability Zones subnet-0dec9610ac3db0224 eu-west-2c (euw2-az1) subnet-07e1c4005375a04c8 eu-west-2a (euw2-az2) subnet-0e51ed9fb91a95164 eu-west-2b (euw2-az3)	

DNS name
af48688251f224c48be71ffe585aeee9-576210719.eu-west-2.elb.amazonaws.com (A Record)

This Classic Load Balancer can be migrated to a next generation load balancer. Migration wizard uses your load balancer's current configurations to create a new load balancer. [Learn more](#)

Distribution of targets by Availability Zone (AZ)
For each enabled Availability Zone, you can view the number of registered instances and their current health states. Selecting any values here will apply the corresponding filter to the Target instances table.

And that is has the DNS name that we can use to access it from

-> af48688251f224c48be71ffe585aaaa9-576210719.eu-west-2.elb.amazonaws.com

et instances	Monitoring	Attributes	Tags
VPC vpc-0b4e83c353f2cb6c6			Date created June 25, 2026,
Availability Zones			
subnet-0dec9610ac3db0224			eu-west-2c (euw2-az1)
subnet-07e1c4005375a04c8			eu-west-2a (euw2-az2)
subnet-0e51ed9fb91a95164			eu-west-2b (euw2-az3)

We can also see that the loadbalancer is linked to 3 subnets and if we check them -> they are public

VPC > Subnets > subnet-0dec9610ac3db0224

C dashboard < subnet-0dec9610ac3db0224 / myapp-vpc-public-eu-west-2c Actions

Global View ⓘ

Filter by VPC

Private cloud

- VPCs
- Subnets
- Route tables
- Internet gateways
- VPN gateways
- VPNs
- IP sets
- Prefix lists
- Gateways
- Connections
- Servers
- Security
- Groups
- Link and Lattice
- Started
- Points
- Services
- Networks
- Services
- Configurations

Details

Subnet ID
subnet-0dec9610ac3db0224

IPv4 CIDR
10.0.6.0/24

Availability Zone
euw2-az1 (eu-west-2c)

Network ACL
acl-0cbe1707b8ee67c27 | myapp-vpc-default

Auto-assign customer-owned IPv4 address
No

IPv6 CIDR reservations
-

Resource name DNS AAAA record
Disabled

Subnet ARN
arn:aws:ec2:eu-west-2:706976333940:subnet/subnet-0dec9610ac3db0224

Available IPv4 addresses
250

Network border group
eu-west-2

Default subnet
No

Customer-owned IPv4 pool
-

IPv6-only
No

DNS64
Disabled

State
Available

IPv6 CIDR
-

VPC
vpc-0b4e83c353f2cb6c6 | myapp-vpc

Auto-assign public IPv4 address
No

Outpost ID
-

Hostname type
IP name

Owner
706976333940

Block Public Access
Off

IPv6 CIDR association ID
-

Route table
rtb-06a9b5c5a8fe9dd9a | myapp-vpc-public

Auto-assign IPv6 address
No

IPv4 CIDR reservations
-

Resource name DNS A record
Disabled

Flow logs | Route table | Network ACL | CIDR reservations | Sharing | Tags

Flow logs

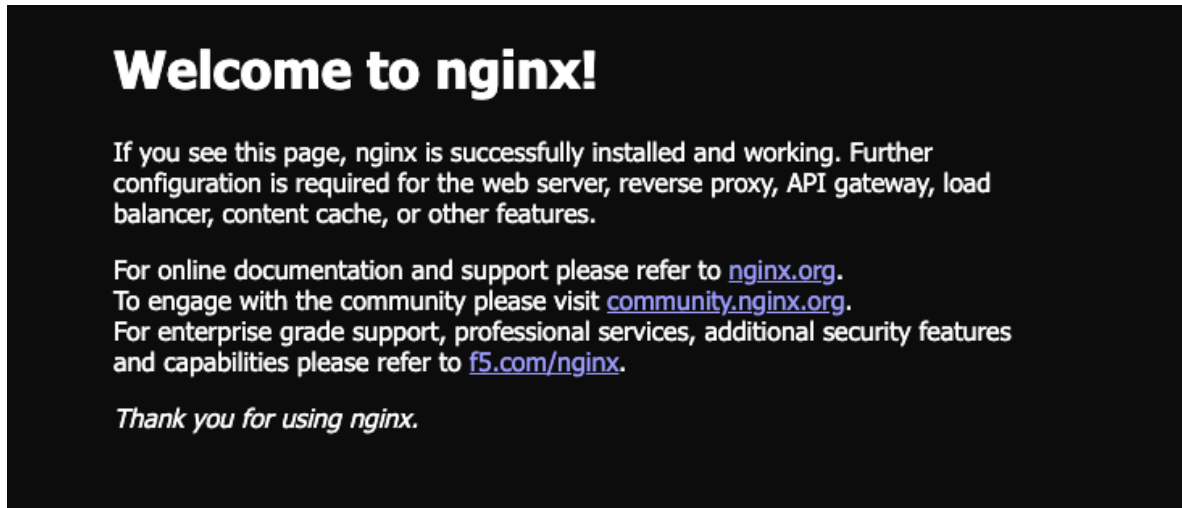
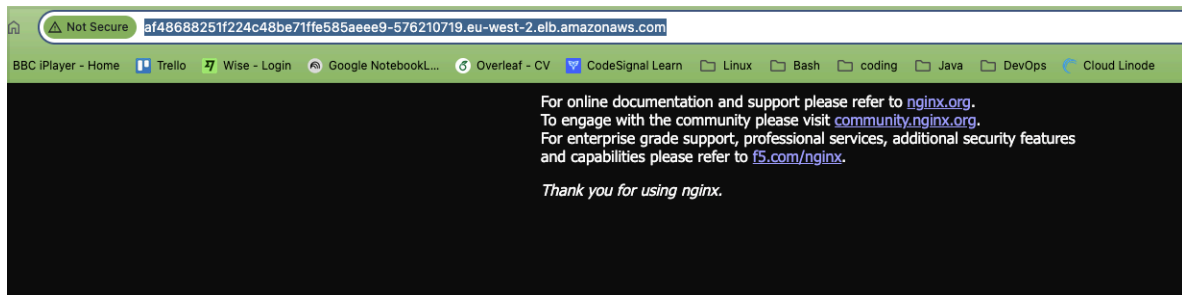
Search

Name	Flow log ID	Traffic type	Destination type	Destination name
No flow logs found				

Actions Create flow log

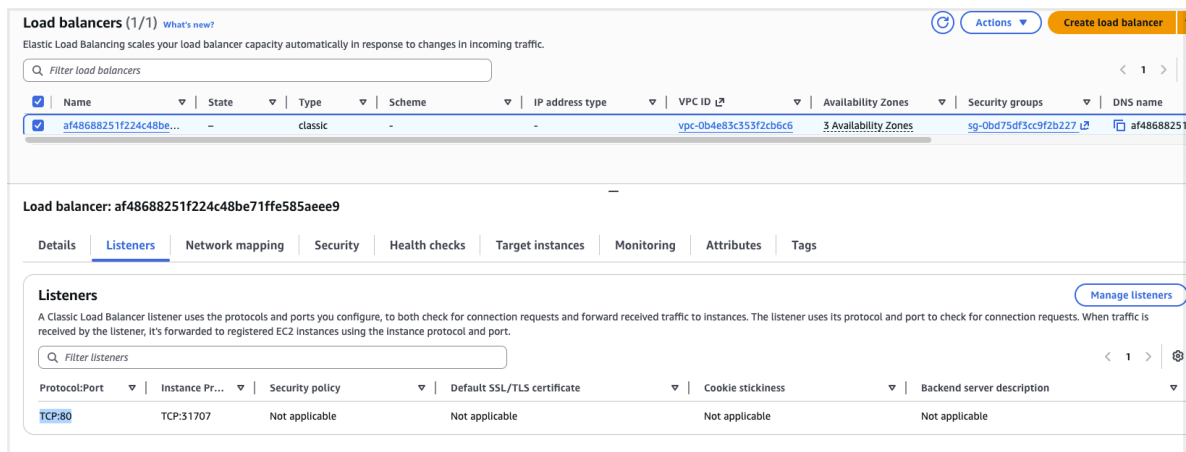
Each public subnet is in one of the 3 AZs.

And if we use the DNS name in the browser



And it works!

If we check the loadbalancer listeners the port 80 is already opened



We didn't have to adjust the security group and open the port. It all happened automatically when the loadbalancer was provisioned.

Destroy all components

Delete the service and the nginx app first as TF doesn't know about them. We created them manually.

1. Delete the service first (this triggers AWS to deprovision the ELB)
kubectl delete service <nginx-service-name>

2. Then delete the deployment

```
kubectl delete deployment <nginx-deployment-name>
```

3. Wait and verify the ELB is fully gone before destroying

```
aws elb describe-load-balancers --query 'LoadBalancerDescriptions[*].  
[LoadBalancerName,VPCId]' --output table
```

4. Once the LB is gone, then run

```
terraform destroy
```

The key step is deleting the service first and waiting. When Kubernetes sees the service deleted, the AWS cloud controller automatically cleans up the ELB, releases the public IPs, and removes the security group it created — all the things that were blocking you today.

-> terraform destroy

-> terraform state list

We should get an empty list

```
> terraform state list
```

```
~/terraform ▶ Module_12/Terraform_chapter-21 +1 !1 ▶
```